LAB 2
DEVELOP SOFTWARE FOR NIOS II PROCESSOR WITH PARALLEL PORT

I. Objective

- Students study Lab 2 after doing all steps in Lab 1. The objective of this Lab is to provide for students the knowledge of programming with Nios II processor using C language. After study Lab 2, students can understand how to program some basic projects with a processor using C language. Besides, students can develop some projects using s parallel ports peripheral such as: LEDs, 7 Segments, buttons, switchs.

II. Preliminary information:

1. Summary of Lab 1

Lab 1 can be summarized into these steps:

- Use DE10-Standard Sytem Builder Tool to create a project with necessary peripherals.
- Use Qsys Tool to configure a system with Nios II processor and PIO IP.
- Create HDL code with Qsys Tool and connect the system with peripherals using VHDL or Verilog on Quartus.
- Use Quartus 18.1 programmer to download .sof file to the development board (DE10- standard) via JTAG interface.
- Use Eclipse Software to program and run c code.

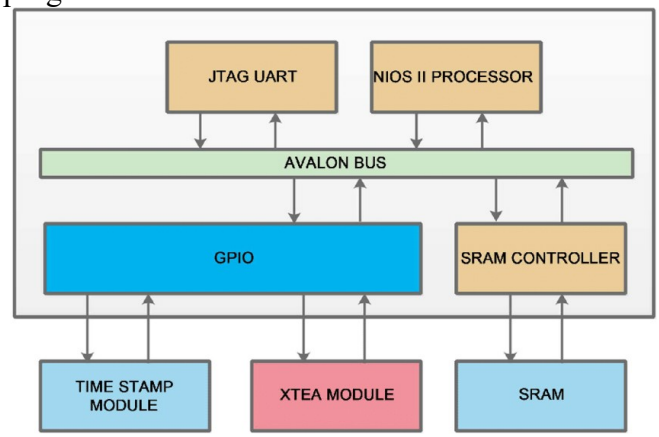


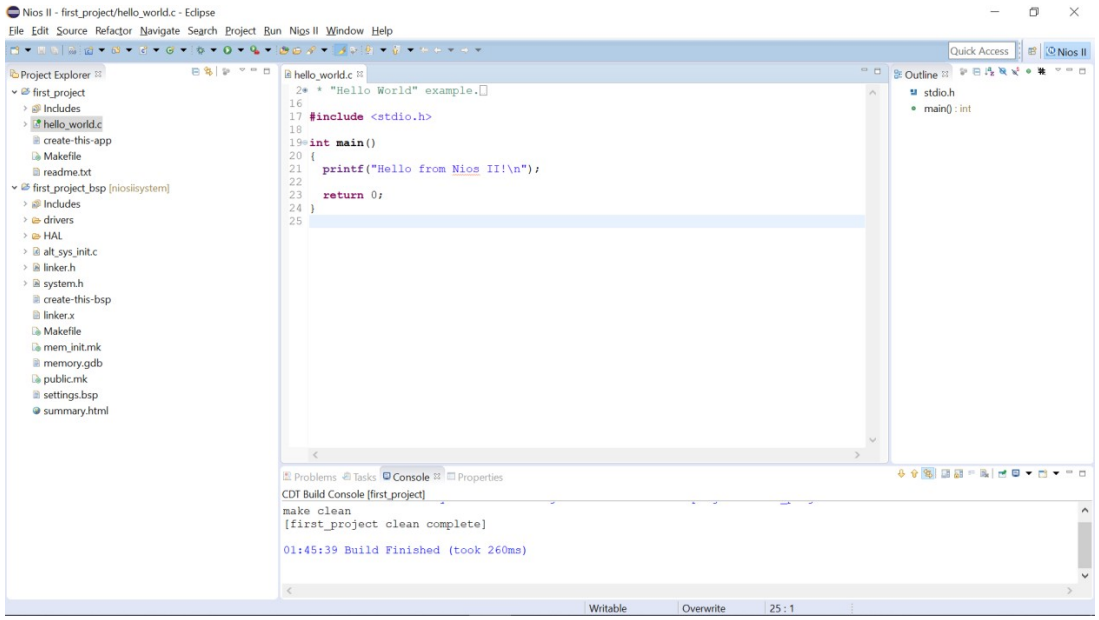
Figure 1: Diagram of a first simple project using NIOS II

Reference:

Chapter 3 - Using the DE10-Standard Board (DE10-Standard User Manual.pdf)

Chapter 4 - DE10-Standard System Builder (DE10-Standard User Manual.pdf)

2. BSP (Board Support Package)

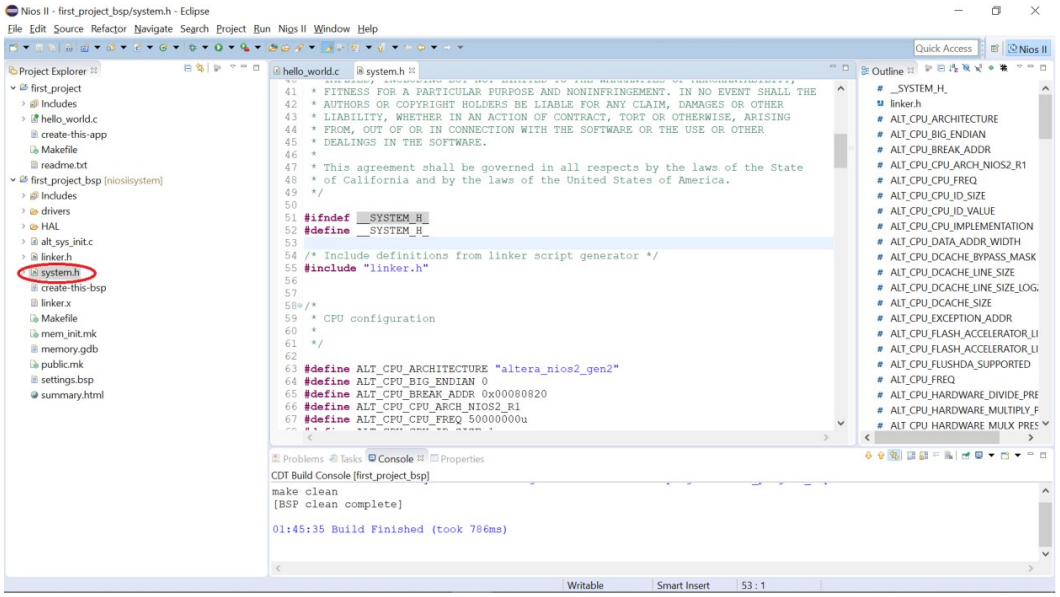


Doing all steps in LAB 1, we have a project with two folders: one is the project folder (the first folder) and two is the BSP (Board Support Package). The BSP is created automatically by Eclipse and Quartus Tool. This folder contains all nescessary libraries used for programming.

From Intel's Guide, we have: Every software application project created in the Nios II Software Build Tools requires an associated BSP (Board Support Package) project. You can create a new BSP every time you create an application project if you like, or you can re-use existing BSP projects to avoid having to establish new BSP settings for your application. The choice is yours. You can even change what BSP project a given application project is associated with to fit your changing project requirements. To help illustrate these options you performed the following steps in the previous lab exercise:

- Created an application and BSP project pair using a built-in software template.
- Ran this project on the development board.
- Created a "blank" application project, associated a pre-existing BSP with it, and added software code to it. We then ran this on the development board.
- Created a brand new BSP project with new project settings, re-associated an existing software application project to it, and ran the application on the development board.

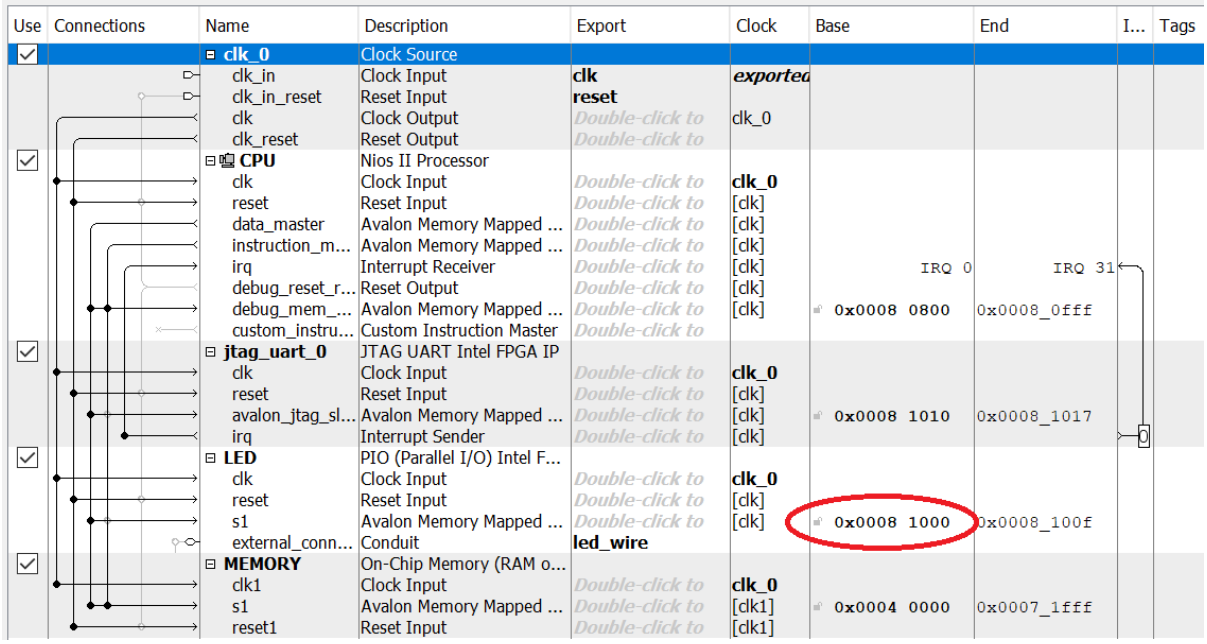
First, in BSP, we considered file system.h. This file contains all important parameters of the system. The frequency of CPU, name and address of each peripheral register, parameters relating to interrupt configuration, … can be found in this file.



For instance, when scrolling down, we can find the code "***#define LED_BASE 0x81000***" This means, the register using for controlling LED is assigned at address. 0x81000.

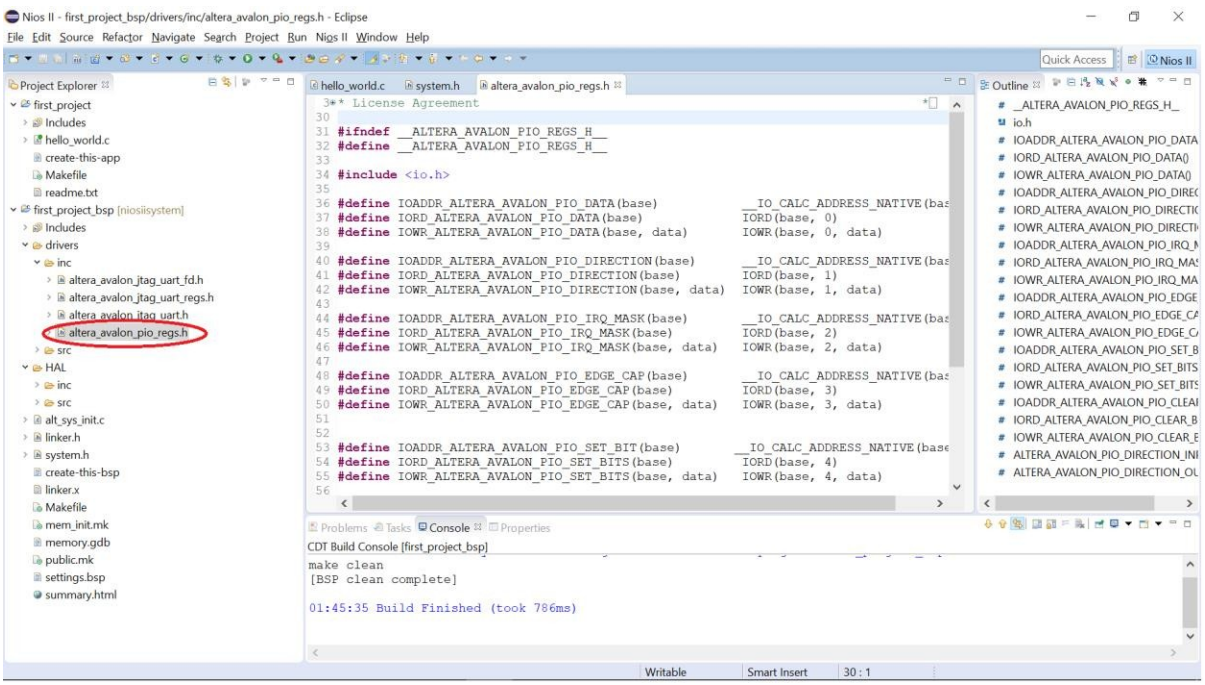
```
#define ALT_MODULE_CLASS_LED altera_avalon_pio
#define LED_BASE 0x81000
#define LED_BIT_CLEARING_EDGE_REGISTER 0
#define LED_BIT_MODIFYING_OUTPUT_REGISTER 0
#define LED_CAPTURE 0
#define LED_DATA_WIDTH 10
#define LED_DO_TEST_BENCH_WIRING 0
#define LED_DRIVEN_SIM_VALUE 0
#define LED_EDGE_TYPE "NONE"
#define LED_FREQ 50000000
#define LED_HAS_IN 0
#define LED_HAS_OUT 1
#define LED_HAS_TRI 0
#define LED_IRQ -1
#define LED_IRQ_INTERRUPT_CONTROLLER_ID -1
#define LED_IRQ_TYPE "NONE"
#define LED_NAME "/dev/LED"
#define LED_RESET_VALUE 0
#define LED_SPAN 16
#define LED_TYPE "altera_avalon_pio"
```

Comparing with Qsys design on Qsys Tool, this address is the address which is assigned by using Qsys Tool. Therefore, if we need to change the address of system design on qsys tool, we have to recreate the BSP.



Use	Connections	Name	Description	Export	Clock	Base	End	I...	Tags
☑		clk_0	Clock Source						
		clk_in	Clock Input	clk	exported				
		clk_in_reset	Reset Input	reset					
		clk	Clock Output	Double-click to	clk_0				
		clk_reset	Reset Output	Double-click to					
☑		CPU	Nios II Processor						
		clk	Clock Input	Double-click to	clk_0				
		reset	Reset Input	Double-click to	[clk]				
		data_master	Avalon Memory Mapped ...	Double-click to	[clk]				
		instruction_m...	Avalon Memory Mapped ...	Double-click to	[clk]				
		irq	Interrupt Receiver	Double-click to	[clk]	IRQ 0	IRQ 31		
		debug_reset_r...	Reset Output	Double-click to	[clk]				
		debug_mem_...	Avalon Memory Mapped ...	Double-click to	[clk]	0x0008 0800	0x0008_0fff		
		custom_instru...	Custom Instruction Master	Double-click to					
☑		jtag_uart_0	JTAG UART Intel FPGA IP						
		clk	Clock Input	Double-click to	clk_0				
		reset	Reset Input	Double-click to	[clk]				
		avalon_jtag_sl...	Avalon Memory Mapped ...	Double-click to	[clk]	0x0008 1010	0x0008_1017		
		irq	Interrupt Sender	Double-click to	[clk]			0	
☑		LED	PIO (Parallel I/O) Intel F...						
		clk	Clock Input	Double-click to	clk_0				
		reset	Reset Input	Double-click to	[clk]				
		s1	Avalon Memory Mapped ...	Double-click to	[clk]	0x0008 1000	0x0008_100f		
		external_conn...	Conduit	led_wire					
☑		MEMORY	On-Chip Memory (RAM o...						
		clk1	Clock Input	Double-click to	clk_0				
		s1	Avalon Memory Mapped ...	Double-click to	[clk1]	0x0004 0000	0x0007_1fff		
		reset1	Reset Input	Double-click to	[clk1]				

Second, in BSP, we considered file ***altera_avalon_pio_regs.h*** directory path */drivers/inc*. This file contains all drivers using to control Parallel Port.



To use these functions, for example:

- **IOWR_ALTERA_AVALON_PIO_DATA(base,data)**
  - IOWR and DATA: means this function used to write data to PIO
  - There are two parameters: base and data. Base is the address of PIO register and data is the writing data.

- **IORD_ALTERA_AVALON_PIO_DATA(base)**
  - IORD and DATA: means this function used to read data from PIO
  - There are one parameters: base. Base is the address of PIO.
- **IOWR_ALTERA_AVALON_PIO_DIRECTION(base,data)**
  - IOWR and DIRECTION: means this function used to write direction to configure PIO become input or output.
  - There are two parameters: base and data. Base is the address of PIO register and data is the writing data.
- **IORD_ALTERA_AVALON_PIO_DIRECTION(base)**
  - IORD and DATA: means this function used to read direction from configured PIO.
  - There are one parameters: base. Base is the address of PIO.

….

3. Controlling Leds

For example, this code is used to blink LED2 with frequency 1Hz.

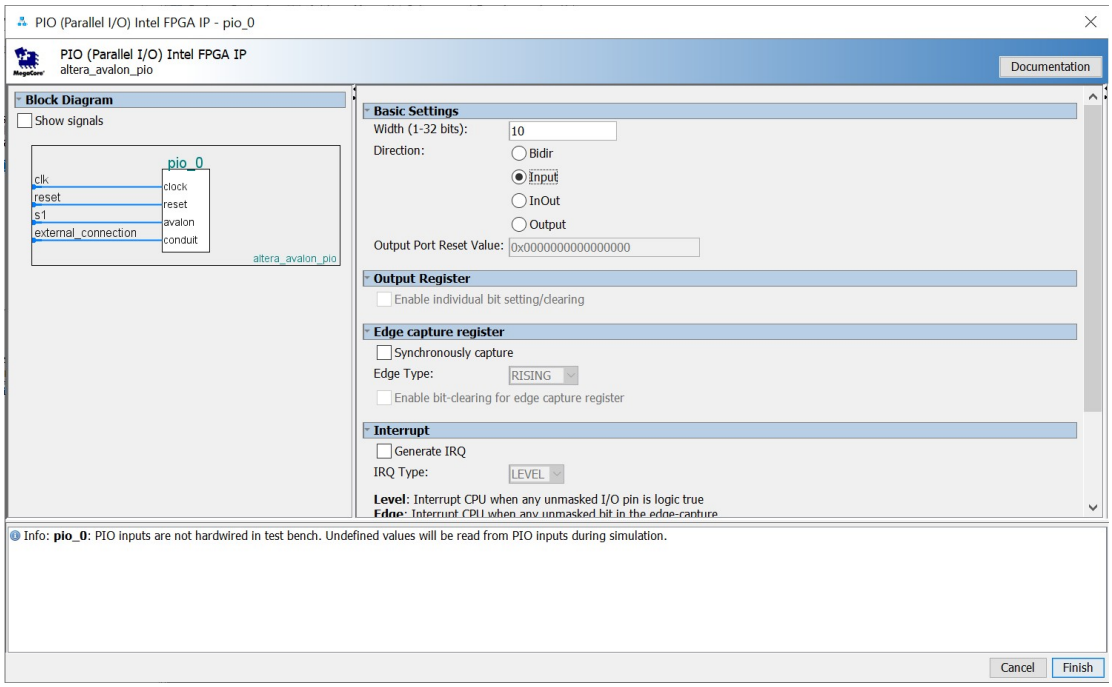
```
#include <stdio.h>
#include <altera_avalon_pio_regs.h>
#include <system.h>
#include <unistd.h>

#define delay 1000000

int main()
{
printf("Hello from Nios II!\n");
/*Loop forever*/
while (1){
IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE,0x02);
usleep(delay);
IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE,0x00);
usleep(delay);
}
return 0;
}
```

- To turn on LED2, we need to write logic 1 to PIN2 which means we need to write value 0x02 in hexadecimal to LED_BASE register.
- To turn off LED2, we need to write logic 0 to PIN2 which means we need to write value 0x00 in hexadecimal to LED_BASE register.
- To use function “IOWR_ALTERA_AVALON_PIO_DATA” and word “LED_BASE”, we need to add 2 libraries system.h and altera_avalon_pio_regs.h.

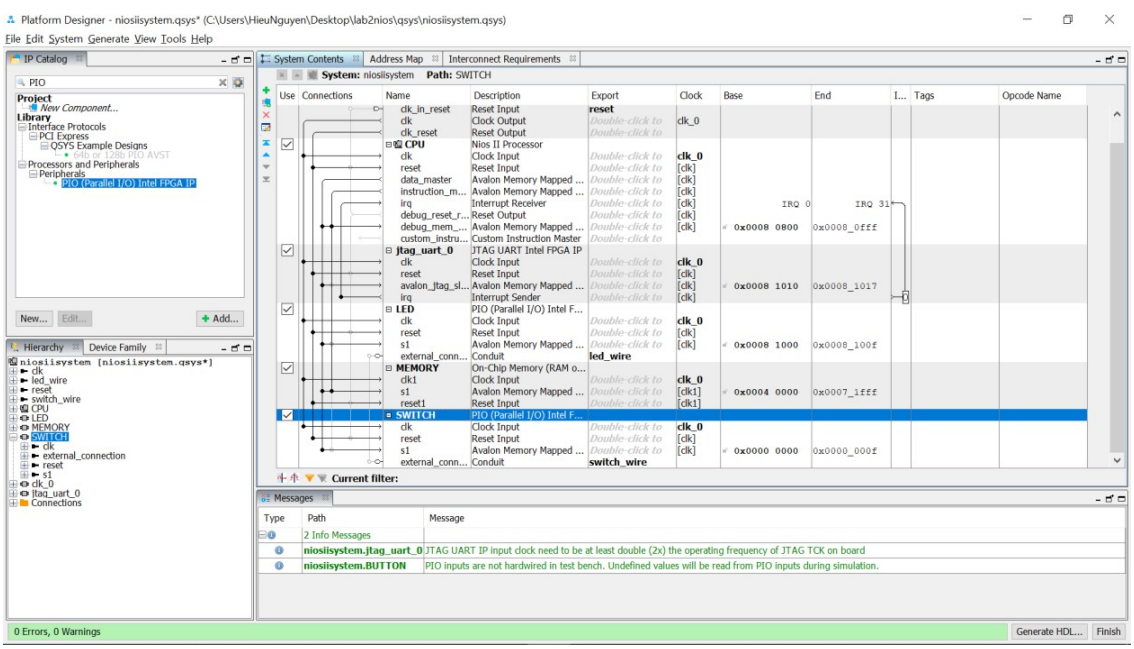
- In the program, we have function usdelay(), this function is a delay function used to create delay time in microseconds. Therefore, the input parameters of this function need to be converted correctly.

4. Using Switch

First, we need to add PIO block using to control switches peripherals. Open Qsys Tool and configure as picture below.



After that, connect switches block to system as picture below. In the picture, the PIO block is renamed into SWITCH, and the external_connection pin is changed into button_wire.



After that, do exactly from step 5 to step 13 in lab 2 to create the project.

For example, this code is used to turn on the LED and print the state of LED to console using the switch on the board.

```
#include <stdio.h>
#include <altera_avalon_pio_regs.h>
#include <system.h>
#include <unistd.h>

int main() {
    while (1) {
        if (IORD_ALTERA_AVALON_PIO_DATA(SWITCH_BASE) == 0x01) {
            IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0x01);
            printf("LED is ON!\n");
        }
        else {
            IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0x00);
            printf("LED is OFF!\n");
        }
    }
    return 0;
}
```

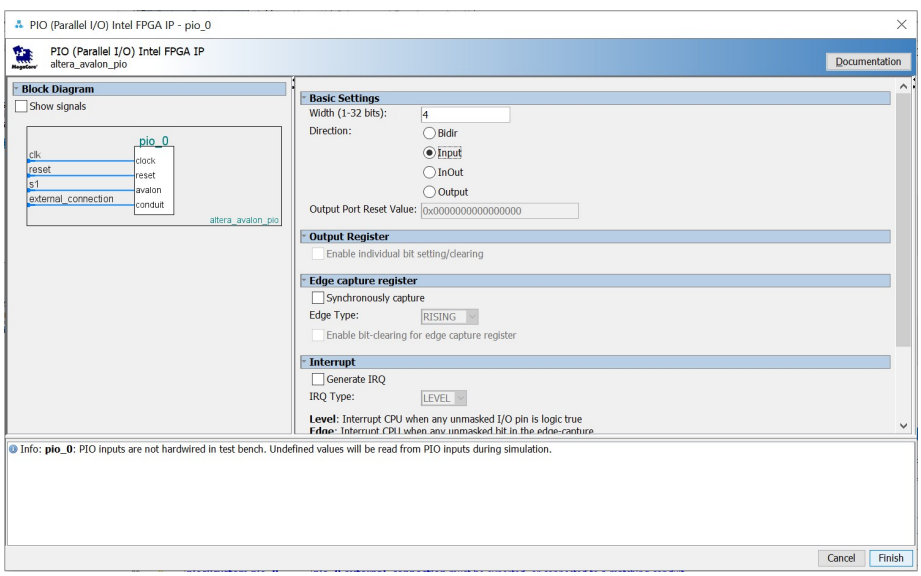
The program reads forever the state of switch 0 and compares with state 1 and 0:

If switch 0's state is 0, turn on LED 0. In contrary case, turn off all LEDs.

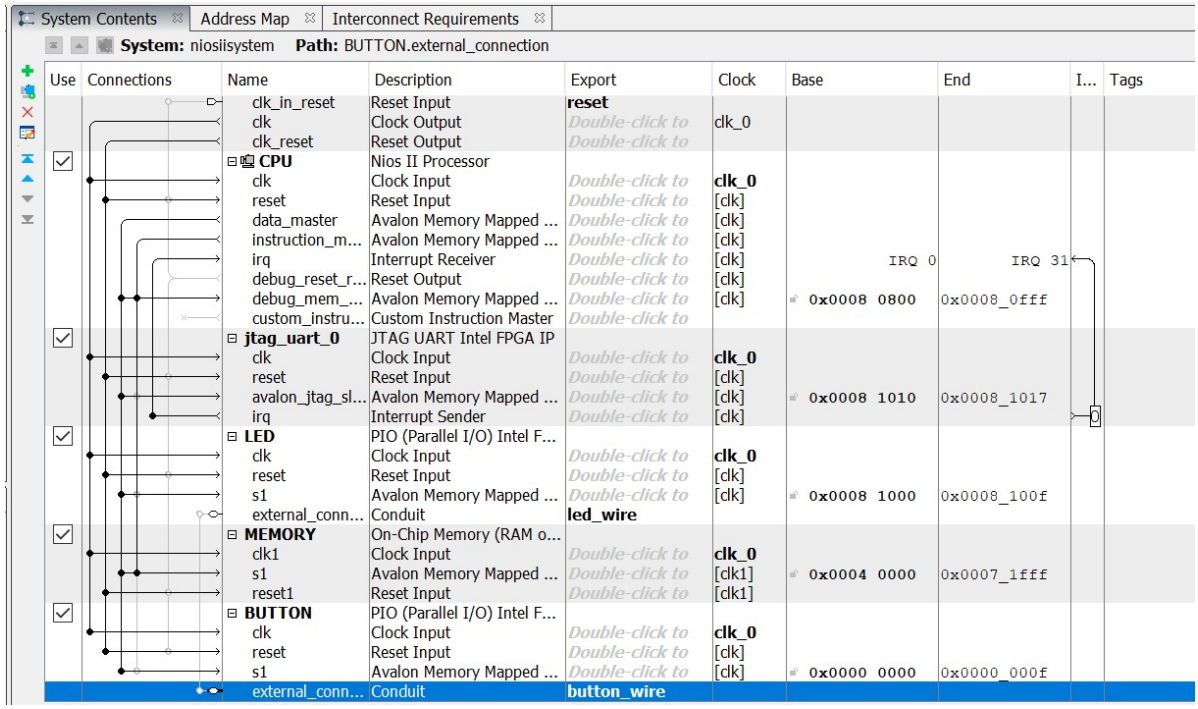
Function IORD_ALTERA_AVALON_PIO_DATA is used to read the state of ALL SWITCH. When switch 0 is set to 1, the value returns from this function is 0x01. If choosing switch 2, the value returning from this function when set to 1 is 0x40.

5. Accesing Buttons

First, we need to add PIO block using to control buttons peripherals. Open Qsys Tool and configure as picture below.



After that, connect buttons block to system as picture below. In the picture, the PIO block is renamed into BUTTON, and the *external_connection* pin is changed into *button_wire*.



After that, do exactly from step 5 to step 13 in lab 1 to create the project.

For example, this code is used to turn on the LED and print the state of LED to console using the buttons on the board.

```
#include <stdio.h>
#include <altera_avalon_pio_regs.h>
#include <system.h>
#include <unistd.h>

#define NONE_PRESSED 0xF
#define DEBOUNCE 100000

int main() {
    int button;
    int led_state = 0;

    while (1) {
        button = IORD_ALTERA_AVALON_PIO_DATA(BUTTON_BASE);
        if (button != NONE_PRESSED) {
            usleep(DEBOUNCE);
            while (button != NONE_PRESSED) {
                button = IORD_ALTERA_AVALON_PIO_DATA(BUTTON_BASE);
            }
            led_state = !led_state;
            IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, led_state);
            printf("LEDs changed state!\n");
        }
    }
    return 0;
}
```

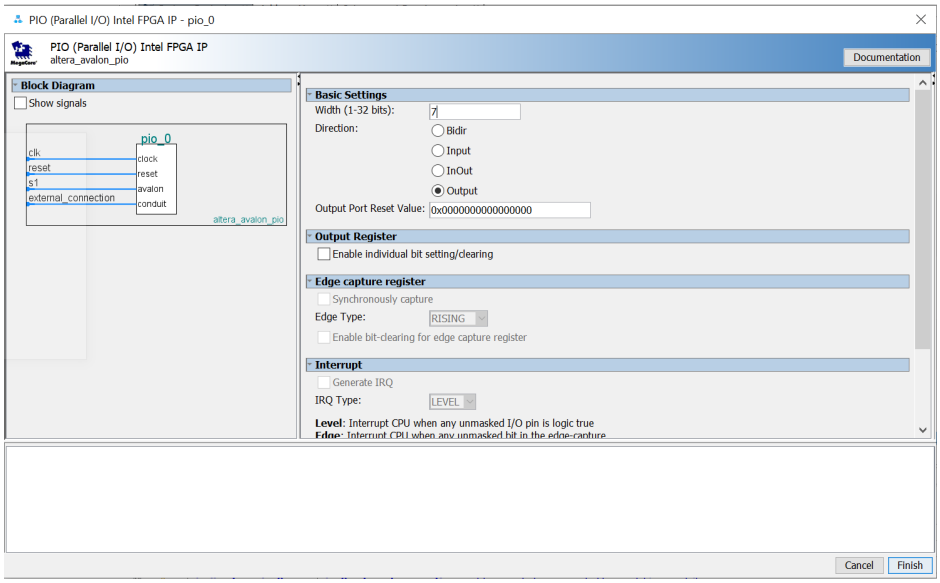
The push button on DE10 board is logic 0 when it is pushed and logic 1 when is not pushed. This means push button is active low. To access push button, we need a debounce logic to make sure the button is pressed. To do that, the code below helps:

```
button = IORD_ALTERA_AVALON_PIO_DATA(BUTTON_BASE);
if (button != NONE_PRESSED) {
    usleep(DEBOUNCE);
    while (button != NONE_PRESSED) {
        button = IORD_ALTERA_AVALON_PIO_DATA(BUTTON_BASE);
    }
```

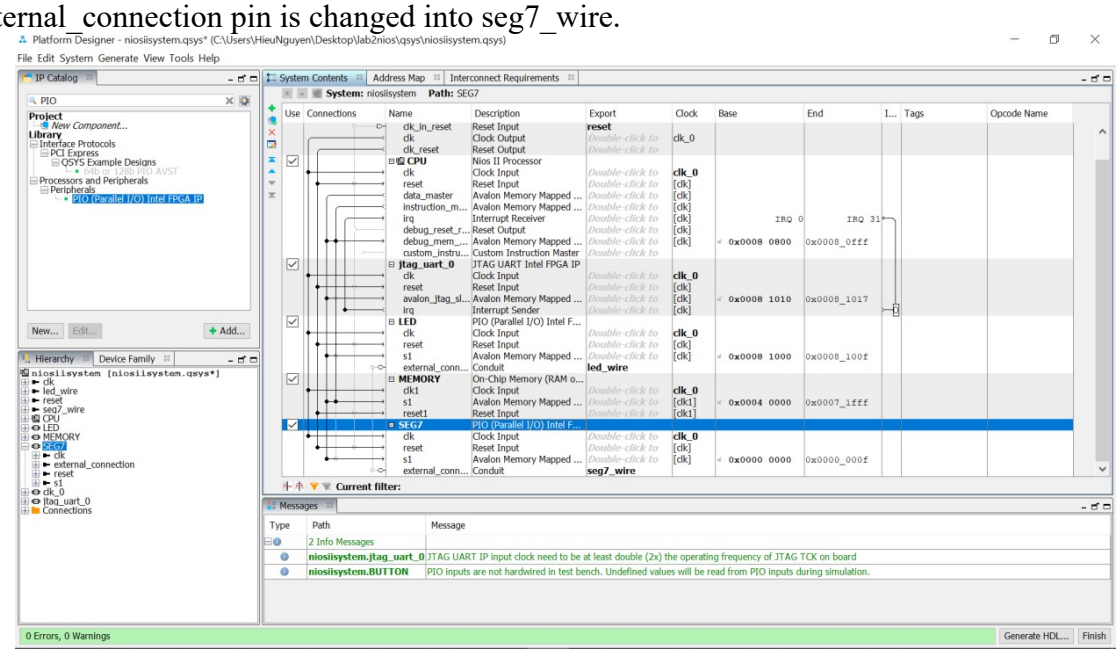
The program in the box is used to change the state of LED0 when a button is pushed.

6. Display 7 Segments Led

First, we need to add PIO block using to control buttons peripherals. Open Qsys Tool and configure as picture below.



Connect buttons block to system as picture below. In the picture, the PIO block is renamed into SEG7, and the external_connection pin is changed into seg7_wire.



III. Experiment

1. Build a Nios II system using LEDs and SWITCHs. Write a C program to do these tasks:

- When switch 1 is set, each led from LED1 to 10 turns on in 500ms and then turn off.
- When switch 2 is set, each led from LED10 to 1 turns on in 500ms and then turn off.
- When two switch are set or not set, turn off all leds.

Bonus question: When changing the state of switch, does the led change their state immediately?

2. Build a Nios II system using 7 SEGMENT LEDs and BUTTONs. Write a C program to do these tasks:

- When reset, all LEDs display 0.
- When the button is pushed, the display on LEDS increases one. (The LEDs count the numbers of pushing times).